# Tema 2.B (segunda parte): Modelado de Requisitos

El modelado de requisitos es una práctica fundamental en el desarrollo de sistemas que implica la creación de representaciones estructuradas y formalizadas de las necesidades y expectativas de los usuarios y stakeholders.

Este proceso transforma descripciones textuales y a menudo ambiguas en artefactos visuales y lógicos que facilitan una comprensión profunda y un diseño robusto. Una de las ventajas más significativas del modelado de requisitos es que facilita una comunicación no ambigua entre todos los stakeholders involucrados en un proyecto.

Además, el modelado permite la verificación automática y el razonamiento sobre los requisitos. Al estar expresados de manera formal, es posible aplicar herramientas y técnicas que detecten inconsistencias, completitud o redundancias dentro del conjunto de requisitos. Esto no solo mejora la calidad del diseño inicial, sino que también sienta las bases para pruebas y validaciones más rigurosas del sistema final.

# Índice


1 **Introducción**

Definición de modelos y sus beneficios en el desarrollo de software.

2 **UML: Lenguaje Unificado de Modelado**

Historia, evolución y una visión general de este lenguaje.

3 **Diagramas de Casos de Uso**

- Casos de uso
- Actores
- Relaciones
  - Asociación
  - Include
  - Extend
  - Generalización

4 **Escenarios de Casos de Uso**

Técnicas para describir detalladamente la interacción entre actores y el sistema.

# Índice


1 **Introducción**

Definición de modelos y sus beneficios en el desarrollo de software.

2 UML: Lenguaje Unificado de Modelado

Historia, evolución y una visión general de este lenguaje.

3 Diagramas de Casos de Uso

- Casos de uso
- Actores
- Relaciones
  - Asociación
  - Include
  - Extend
  - Generalización

4 Escenarios de Casos de Uso

Técnicas para describir detalladamente la interacción entre actores y el sistema.

# ¿Qué es un Modelo?

Un modelo es una **representación simplificada de la realidad** que nos permite comprender, analizar y comunicar aspectos complejos de un sistema. Los modelos abstraen los detalles innecesarios y destacan las características esenciales, facilitando la comprensión del sistema por parte de todos los participantes del proyecto.

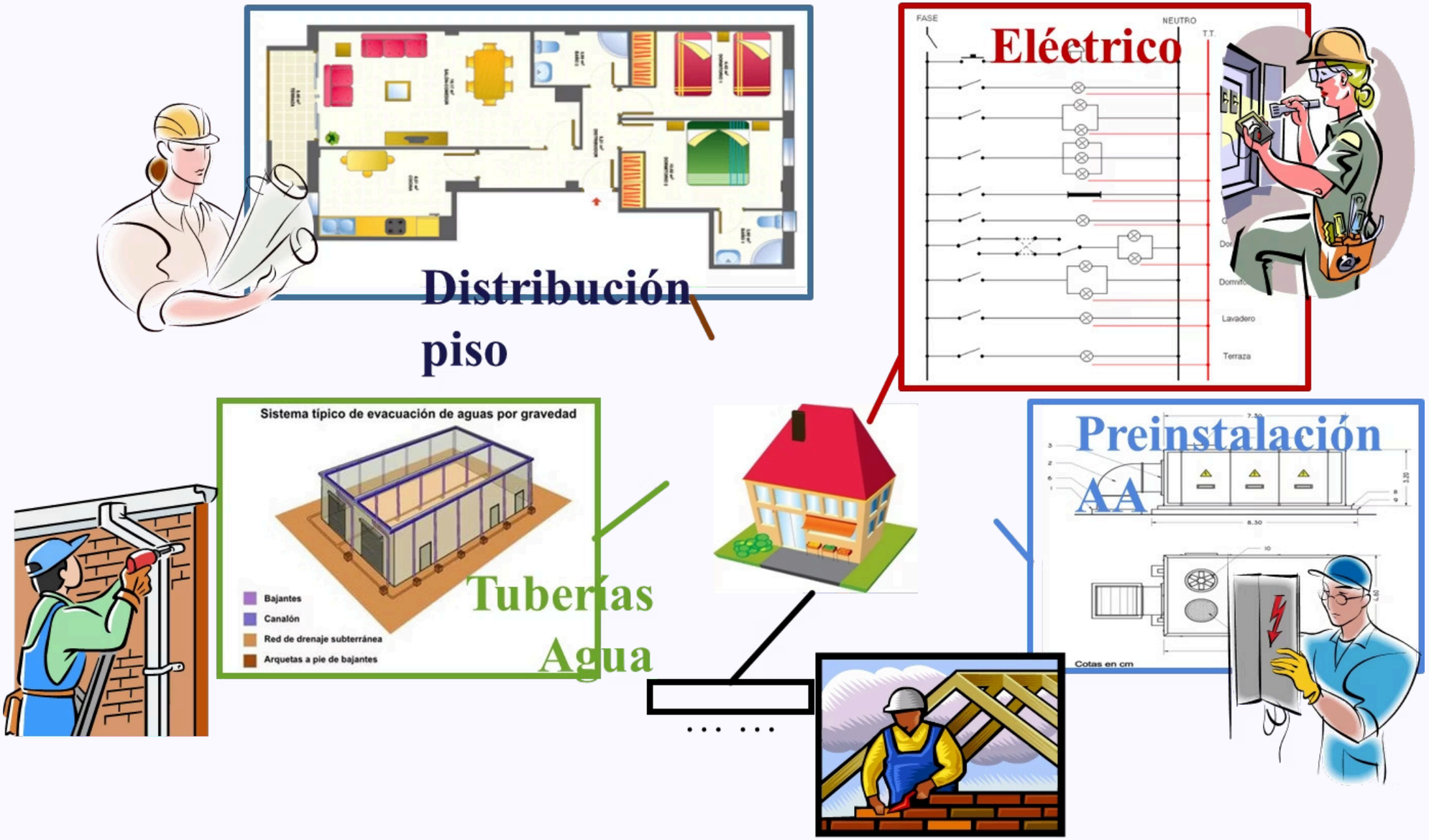
Al igual que un arquitecto utiliza planos para diseñar un edificio antes de construirlo, en ingeniería del software utilizamos modelos para visualizar y planificar sistemas antes de implementarlos.

> *"Modeling, in the broadest sense, is the cost-effective use of something in place of something else for some cognitive purpose. It allows us to use something that is simpler, safer or cheaper than reality instead of reality for some purpose. A model represents reality for the given purpose; the model is an abstraction of reality in the sense that it cannot represent all aspects of reality. This allows us to deal with the world in a simplified manner, avoiding the complexity, danger and irreversibility of reality.“*
>
> "*The Nature of Modeling*." Jeff Rothenberg in Artificial Intelligence, Simulation, and Modeling

# ¡Esto no es un plátano!

Aunque veas la imagen de un plátano, esta representación en sí misma no es el objeto real. Es una abstracción, una forma de entender y comunicar el concepto de un plátano sin tener el fruto físico. De manera similar, los modelos de requisitos no son el software final, sino herramientas para conceptualizarlo.

# ¿Por qué utilizar modelos?




ⓘ Los modelos son imprescindibles en la ingeniería del software al igual que en el resto de ingenierías

# ¿Por qué utilizar modelos?

### Comunicación Efectiva

Proporcionan un lenguaje común y no ambiguo entre desarrolladores, clientes y stakeholders, reduciendo malentendidos en los requisitos.

### Comprensión Profunda

Facilitan la visualización de la estructura y comportamiento de sistemas complejos, permitiendo identificar problemas antes de la implementación.

### Documentación Continua

Sirven como documentación viva y más estable que descripciones textuales, útil para mantenimiento futuro y onboarding.

### Validación Temprana

Permiten detectar errores e inconsistencias en las primeras fases del proyecto, reduciendo costes de corrección posteriores y utilizando herramientas automáticas para verificación.

### Reutilizabilidad y Expresividad

Son una solución general y expresiva, facilitando la reutilización en distintos contextos y la resolución de problemas similares.

### Análisis y Razonamiento

Permiten razonar acerca del sistema, extraer conocimiento valioso y verificar propiedades importantes mediante análisis formales.

# Índice


1 **Introducción**

Definición de modelos y sus beneficios en el desarrollo de software.

2 **UML: Lenguaje Unificado de Modelado**

Historia, evolución y una visión general de este lenguaje.

3 **Diagramas de Casos de Uso**

- Casos de uso
- Actores
- Relaciones
  - Asociación
  - Include
  - Extend
  - Generalización

4 **Escenarios de Casos de Uso**

Técnicas para describir detalladamente la interacción entre actores y el sistema.

# UML: Unified Modeling Language

UML (Unified Modeling Language) es un **lenguaje de modelado visual** estándar en la industria del software. Fue desarrollado en los años 90 por Grady Booch, James Rumbaugh e Ivar Jacobson para unificar diferentes notaciones de modelado existentes y ha ido evolucionando hasta nuestros días.

UML proporciona un conjunto rico de diagramas que permiten modelar diferentes aspectos de un sistema software, desde su estructura estática hasta su comportamiento dinámico. Es ampliamente utilizado en metodologías de desarrollo orientadas a objetos y es **reconocido internacionalmente como estándar ISO**.

UNIFIED MODELING LANGUAGE ™

**¿Por qué usar UML?**

- Estándar internacional
- Independiente del lenguaje de programación
- Soportado por múltiples herramientas
- Mejora la calidad del software

# La Evolución de UML

1

### 1994-1996: La "Guerra de Métodos"

Grady Booch y James Rumbaugh inician la unificación. Ivar Jacobson se une, y publican UML 0.9, cambiando de "Método" a "Lenguaje Unificado de Modelado".

2

### 1997: Estandarización OMG

Se presenta UML 1.0. En noviembre, el Object Management Group (OMG) adopta oficialmente UML 1.1, estableciéndolo como estándar formal.

3

### 1999-2003: Mejoras de UML 1.x

UML 1.3 (revisión de inconsistencias), UML 1.4 (mejoras en extensibilidad), y UML 1.5 (introducción de "Action Semantics").

4

### 2005: La Gran Evolución (UML 2.0)

Reescritura masiva del lenguaje y adopcion como norma ISO (Organización Internacional de Normalización).

5

### 2007-2011: Consolidación de UML 2.x

Versiones 2.1.1, 2.1.2, 2.2, 2.3 y 2.4.1. Enfocadas en correcciones, mejoras en diagramas de componentes y despliegue, y clarificaciones semánticas.

6

### 2015: Reorganización (UML 2.5)

Gran reorganización para hacerla más legible.

7

### 2017: Versión Actual (UML 2.5.1)

La versión vigente, que continúa el mantenimiento.

# Tipos de Diagramas UML

## Diagramas de Comportamiento (dinámicos)

Representan el comportamiento dinámico del sistema

- Diagrama de Casos de Uso
- Diagrama de Actividades
- Diagrama de Secuencia
- Diagrama de Estados
- Diagrama de Comunicación

## Diagramas Estructurales (estáticos)

Representan la estructura estática del sistema

- Diagrama de Clases
- Diagrama de Objetos
- Diagrama de Componentes
- Diagrama de Despliegue
- Diagrama de Paquetes

# Índice


1 **Introducción**

Definición de modelos y sus beneficios en el desarrollo de software.

2 **UML: Lenguaje Unificado de Modelado**

Historia, evolución y una visión general de este lenguaje.

3 **Diagramas de Casos de Uso**

- Casos de uso
- Actores
- Relaciones
  - Asociación
  - Include
  - Extend
  - Generalización

4 **Escenarios de Casos de Uso**

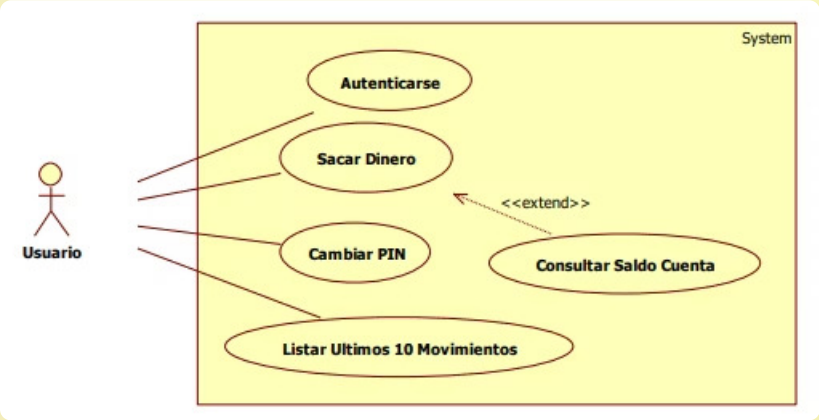
Técnicas para describir detalladamente la interacción entre actores y el sistema.

# Diagramas de Casos de Uso

Los diagramas de casos de uso son un mecanismo fundamental para la **especificación de requisitos**, basado en el modelado de escenarios de uso de un producto software.

Estos diagramas capturan los **requisitos funcionales** de un sistema desde la perspectiva del usuario, describiendo las interacciones entre el sistema y los elementos externos, conocidos como actores. Su propósito principal es indicar qué hace el sistema, y no cómo lo hace.

Estos diagramas son especialmente útiles en las fases iniciales del proyecto como **visión de alto nivel** del sistema que sirve para establecer el alcance y facilitar la comunicación con los stakeholders.



# Elementos Fundamentales

### Casos de Uso

Describen funcionalidades específicas del sistema desde el punto de vista del usuario. Se representan como óvalos con el nombre de la acción.

### Actores

Representan roles externos que interactúan con el sistema. Pueden ser personas, otros sistemas o dispositivos hardware.

### Relaciones

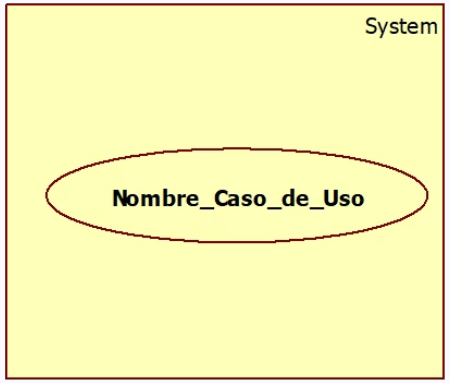
Conectan actores con casos de uso (asociación), entre casos de uso (inclusión, extensión, generalización), y entre actores (generalización).

### Límite del Sistema

Un rectángulo que delimita el alcance del sistema, conteniendo todos los casos de uso pero dejando los actores fuera.

# 1. Casos de Uso

Un caso de uso representa una **funcionalidad** completa del sistema que proporciona valor al actor. Debe ser iniciado por un actor y tener un resultado observable.




## Reglas de estilo

- Usar verbos en infinitivo
- Ser específico y conciso
- Evitar términos técnicos cuando sea posible
- Reflejar la perspectiva del usuario

**Ejemplos:** Realizar Compra, Consultar Saldo, Generar Informe, Registrar Usuario

# 2. Actores

Un actor representa un rol que interactúa con el sistema, no una persona específica. Una misma persona puede desempeñar múltiples roles, y múltiples personas pueden compartir el mismo rol.

Un actor puede ser:

- **Usuarios** que interactúan con una aplicación.
- Otras **aplicaciones** con las que existe una interfaz (p.e., pasarela de pago, etc.)
- **Elementos físicos** (p.e., sensores, etc.)

### Tipos de Actores

#### Primarios

Inician la interacción y obtienen valor directo del sistema

#### Secundarios

Proporcionan servicios al sistema o reciben información de él

# 3. Relaciones

Los diagramas de casos de uso definen cómo los actores interactúan con el sistema y cómo se estructuran las funcionalidades internas. Estas interacciones se representan mediante diversos tipos de relaciones:

### Asociación

Conexión entre un actor y un caso de uso, indicando que el actor participa o inicia esa funcionalidad.

### Include

Un caso de uso incorpora obligatoriamente la funcionalidad de otro caso de uso, reutilizando comportamientos comunes.

### Extend

Un caso de uso añade una funcionalidad opcional al comportamiento de otro caso de uso bajo condiciones específicas.

### Generalización

Define una relación de herencia entre casos de uso o entre actores, donde el elemento especializado hereda las características del general.

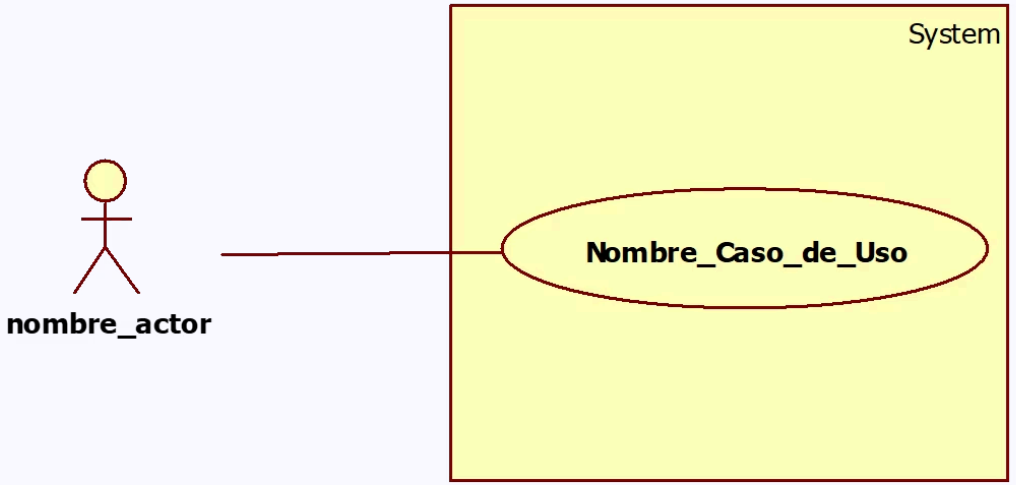
# 3.1 Asociación

La relación de asociación es la conexión fundamental entre un actor y un caso de uso.

Permite especificar la participación de un actor en una funcionalidad específica del sistema, indicando que el actor inicia, participa o se beneficia de la misma.

## Consideraciones clave

- Se representa con una línea continua entre el actor y el caso de uso.
- Establece claramente quién interactúa con qué parte del sistema.



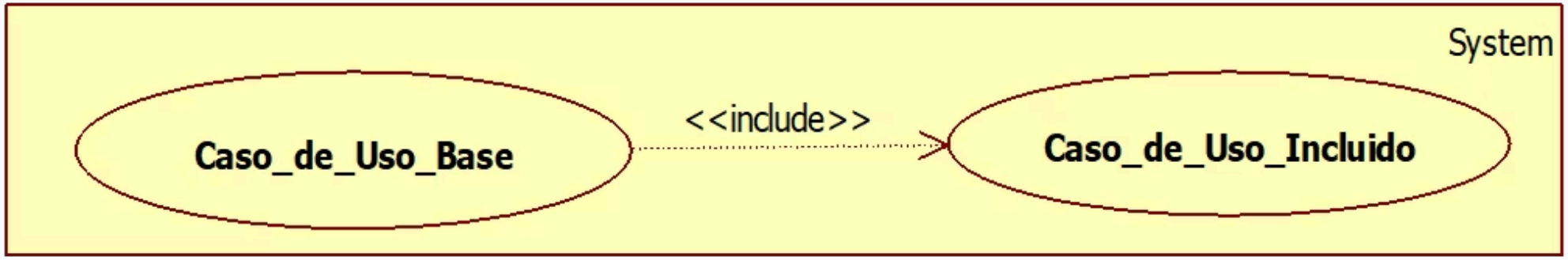
# 3.2 Include

La relación include se utiliza cuando un caso de uso siempre necesita la funcionalidad de otro caso de uso para completarse. Es una forma de reutilizar comportamiento común y evitar duplicación.

### Características

- El caso de uso incluido es **parte esencial** del caso de uso base
- Siempre que se ejecuta el caso de uso base, se ejecuta también el caso de uso incluido
- Etiquetada con `<<include>>`. Se representa con una flecha discontinua que va desde el caso base al incluido

Ejemplo: "Procesar Pedido" siempre debe "Pagar" antes de confirmar la venta. Esta verificación es un paso esencial que se reutiliza en múltiples casos de uso.



# ¿Cuándo usar la relación «Include»?

La relación `<<include>>` es esencial para modelar la **reutilización** y modularidad de las funcionalidades. Además, también se puede emplear en situaciones específicas para **mejorar la claridad del modelado**.

### Reutilización de Comportamiento

Cuando una funcionalidad es un paso obligatorio y común para varios casos de uso base, se extrae a un caso de uso incluido para evitar la duplicación y promover la coherencia.

### Mejorar la calidad del modelado

- **Interacción con Actores Externos Específicos:** Si una parte de un caso de uso siempre requiere interactuar con un actor externo (ej. pasarela de pago), esta interacción puede encapsularse en un caso de uso incluido.
- **Detallar comportamientos en una jerarquía de herencia:** Si en una jerarquía de herencia de casos de uso se quieren detallar inclusiones específicas en ciertos casos de uso de dicha jerarquía.

⊗ Cuidado, no confundir cualquier paso interno de un caso de uso (p.e., en "Comprar entradas" la acción de seleccionar asientos) con un caso de uso incluído que tiene suficiente entidad como caso de uso (p.e., en "Comprar entradas" la acción de pagar)

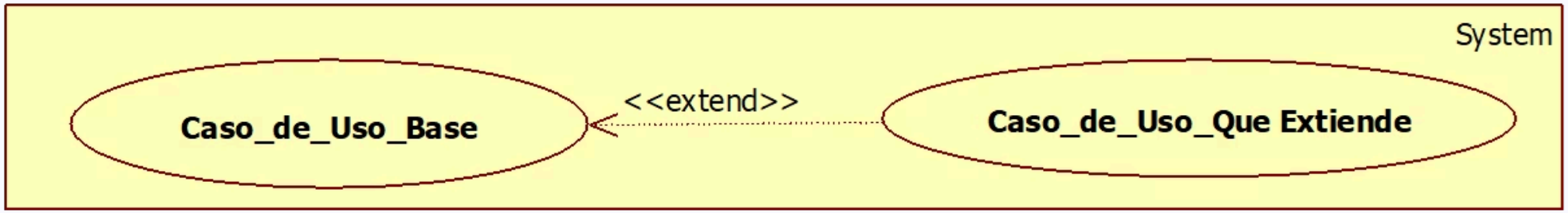
# 3.3 Extend

La relación extend se utiliza cuando un caso de uso puede añadir comportamiento adicional a otro de forma opcional, bajo ciertas condiciones específicas.

### Características

- El caso de uso extensor es **parte opcional** del caso de uso base
- Cuando se ejecuta el caso de uso base, a veces se ejecuta el caso de uso extensor y otras no
- Etiquetada con `<<extend>>`. Se representa con una flecha discontinua que va desde el caso extensor al base

Ejemplo: "Aplicar Descuento" extiende a "Realizar Compra" solo cuando el cliente tiene un cupón válido. La compra puede completarse aplicando descuentos o no.



# ¿Cuándo usar la relación «Extend»?

La relación `<<extend>>` es clave para aportar **flexibilidad** al sistema. Permite insertar comportamientos adicionales en un caso de uso existente sin alterar su flujo principal, modelando situaciones que son opcionales o condicionales.

### Comportamiento Opcional

Se utiliza cuando una funcionalidad ofrece un valor añadido al usuario pero no es obligatoria para completar el objetivo principal del caso de uso base. El sistema base funciona perfectamente sin ejecutar esta extensión

### Flujos alternativos y manejo de errores

La funcionalidad de extensión se ejecuta solo si se cumplen determinadas condiciones o eventos (p. ej., un error o una validación). Esto mantiene el caso base limpio y enfocado en su flujo principal.

# Diferencias: Include vs Extend

## Include

- **Siempre** se ejecuta
- Funcionalidad **obligatoria**
- Reutilización de comportamiento común
- El caso base no puede completarse sin él
- Flecha desde base → incluido

Ejemplo: "Realizar Pago" include "Validar Tarjeta"

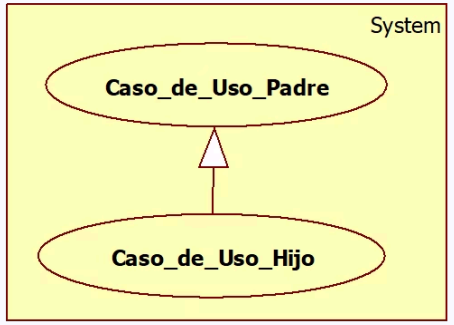
## Extend

- **Condicionalmente** se ejecuta
- Funcionalidad **opcional**
- Variaciones del flujo principal
- El caso base puede completarse sin él
- Flecha desde extensor → base

Ejemplo: "Envolver para regalo" extend "Comprar"

# 3.4.1 Generalización de casos de uso

La generalización permite que casos de uso específicos hereden el comportamiento de un caso de uso más general. Se utiliza cuando varios casos de uso comparten estructura y comportamiento común.




### Características

- Representa una relación "es un tipo de"
- Los casos hijo heredan comportamiento del padre y añaden comportamiento específico.
- El caso de uso general puede ser abstracto si solo sirve para agrupar comportamiento común y no se ejecuta directamente.
- Se dibuja con flecha continua con triángulo

Ejemplo: "Pagar" es el caso de uso padre, mientras que "Pagar con Tarjeta" es el caso de uso hijo que hereda la funcionalidad de su padre y añade la validación bancaria.

# 3.4.2 Generalización de actores

Al igual que los casos de uso, los actores también pueden organizarse en jerarquías de generalización. Un actor hijo hereda todas las asociaciones y características del actor padre, y además se especializa con asociaciones exclusivas que no están disponibles para el actor padre."

### Ventajas

- Reduce redundancia en el diagrama
- Facilita el mantenimiento del modelo
- Refleja relaciones naturales entre roles
- Permite especialización de comportamiento

Ejemplo: "Usuario" puede ser el actor padre y tener relación con ciertos casos de uso (p.e., "Ver perfil"). El actor hijo "Administrador" hereda el comportamiento de "Usuario" y añade relaciones con nuevos casos de uso (p.e., borrar usuarios).

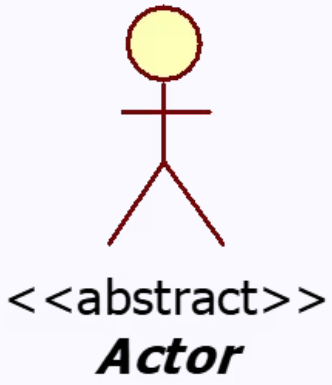
# Actores Abstractos

Un actor abstracto es un actor que no existe como entidad independiente en el mundo real, sino que sirve únicamente para factorizar comportamiento común entre varios actores concretos.

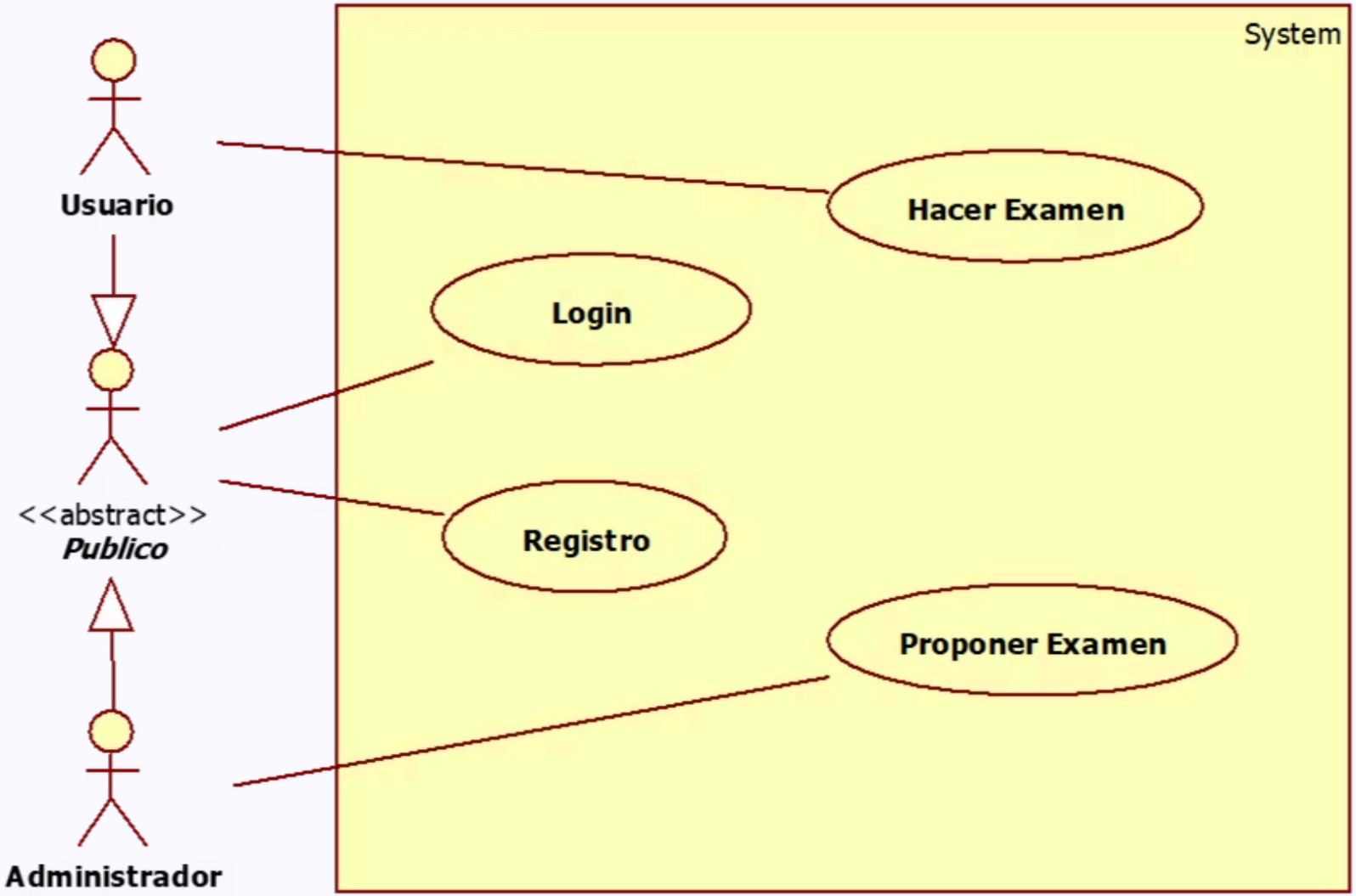
## Características

- Los actores abstractos se utilizan en jerarquías de generalización para agrupar características compartidas. En la práctica no interactúan directamente con el sistema, solo sus hijos concretos.
- Se representan igual que los actores normales, pero su nombre se escribe en *cursiva* o se añade la etiqueta `{abstract}`.

Ejemplo: "Usuario" puede ser el actor padre que realmente no existe en el sistema y tiene como actores hijos "Cliente" y "Administrador". Ambos heredan las capacidades básicas de "Usuario", pero cada uno se relaciona con nuevos casos de uso.



# Actores abstractos: Ejemplo



# Diagramas de Casos de Uso: Resumen

### Casos de Uso

Definen las funcionalidades que el sistema debe proporcionar, representando los objetivos del usuario y las interacciones del sistema.

### Actores

Representan roles externos (humanos o sistemas) que interactúan con el sistema. Pueden ser concretos o abstractos para gestionar la herencia de comportamientos.

### Tipos de Relaciones

- **Asociación:** Conexión básica entre actor y caso de uso.
- **Include:** Reutilización de un comportamiento obligatorio.
- **Extend:** Adición opcional de comportamiento bajo condiciones.
- **Generalización:** Herencia de comportamiento entre casos de uso o actores.

### Limites del sistema

Delimita el alcance del sistema, definiendo todos los casos de uso que se van a realizar dentro de la aplicación y los actores que van a interactuar con dichos casos de uso.

# Índice


1 **Introducción**

Definición de modelos y sus beneficios en el desarrollo de software.

2 **UML: Lenguaje Unificado de Modelado**

Historia, evolución y una visión general de este lenguaje.

3 **Diagramas de Casos de Uso**

- Casos de uso
- Actores
- Relaciones
  - Asociación
  - Include
  - Extend
  - Generalización

4 **Escenarios de Casos de Uso**

Técnicas para describir detalladamente la interacción entre actores y el sistema.

# Escenarios de Casos de Uso

Un escenario es una secuencia concreta y detallada de pasos que describe un "hilo de uso" particular a través de un caso de uso. Representa una ejecución específica, mostrando las interacciones entre un actor y el sistema en un contexto determinado.

### Perspectiva del Actor

Cada escenario se narra desde el punto de vista de un actor (persona o dispositivo) que interactúa con el software. Esto ayuda a comprender sus motivaciones y las respuestas esperadas del sistema.

#### Escenarios de Éxito

Describen el "camino feliz", donde todas las acciones se completan sin inconvenientes, logrando el objetivo principal del caso de uso.

**Ejemplo:** Alta de un alumno en matrícula sin incidencias.

#### Escenarios de Fracaso

Detallan situaciones donde el objetivo del caso de uso no se alcanza, generalmente debido a errores, datos incorrectos o condiciones no cumplidas.

**Ejemplo:** Intento de matrícula con asignaturas cerradas.

# Descripción del escenario de caso de uso

Cada caso de uso debe documentarse con una descripción textual, que puede ser:

### Alto nivel o breve

Descripciones concisas que resumen el escenario de éxito típico. Son vagas, evitan detalles de diseño y se redactan rápidamente para una visión general.

**Ejemplo:** Procesar venta: Un cliente llega a la caja con los productos que desea comprar. El cajero registra los productos y el cliente paga. El sistema registra la venta y emite un recibo.

### Extendido, expandido o completo

Ofrecen una descripción pormenorizada que desglosa los eventos paso a paso. No solo cubren el escenario principal de éxito, sino que también documentan exhaustivamente todos los escenarios alternativos.

Son fundamentales para sistemas complejos o funcionalidades críticas, ya que proporcionan la claridad necesaria para el diseño, desarrollo y pruebas, minimizando ambigüedades y errores.

# Descripción extendida de caso de uso

La descripción extendida de un caso de uso proporciona detalles completos sobre cómo se ejecuta un caso de uso. Es un documento textual estructurado que complementa el diagrama visual.

## Principales elementos

### Identificación

Nombre, identificador único y breve descripción del propósito

### Actores

Actores involucrados en la ejecución del caso de uso

### Precondiciones

Estado que debe cumplir el sistema antes de que el caso de uso pueda iniciarse

### Flujo Principal

Secuencia numerada de pasos que describe la interacción normal entre actor y sistema

### Flujos Alternativos

Variaciones del flujo principal que representan caminos diferentes (caminos válidos o errores)

### Postcondiciones

Estado del sistema después de completar exitosamente el caso de uso

# Ejemplo de descripción extendida (1/3)

- **Nombre:** Contratar vuelos (#17)
- **Descripción**: El Turista introduce los datos del viaje (origen, destino y fechas) para que el sistema valide la disponibilidad; posteriormente, facilita su tarjeta de crédito, y tras la autorización del cobro por parte de la Pasarela de Comercio Electrónico, el sistema emite el billete final.
- **Actores:** Turista y pasarela de comercio electrónico
- **Tipo:** Primario
- **Precondiciones:** Usuario Registrado
- **Postcondiciones**: Billete emitido, cobro realizado y plaza reservada

# Ejemplo de descripción extendida (2/3)

- **Curso típico:**

| Turista | Sistema | Pasarela Comercio Electrónico |
|---|---|---|
| 1. Rellenar de ciudad origen y destino | | |
| | 2. Validar origen y destino | |
| 3. Rellenar fecha, hora y clase | | |
| | 4. Validar fecha, hora y clase | |
| | 5. Comprobar ocupación | |
| 6. Rellenar tarjeta de crédito | | |
| | 7. Validar tarjeta de crédito | |
| | | 8. Autorizar transacción |
| | 9. Emitir billete | |

# Ejemplo de descripción extendida (3/3)

- **Curso alternativo A:** La Pasarela rechaza el pago *(Este flujo comienza tras el paso 7, sustituyendo al paso 8 del curso normal)*

| Turista | Sistema | Pasarela Comercio Electrónico |
|---|---|---|
| | | 8a. Rechazar transacción (Saldo insuficiente / Datos incorrectos) |
| | 9a. Notificar error de pago y ofrecer reintentar | |
| 10a. Introducir nueva tarjeta de crédito (El flujo vuelve al paso 7) | | |